

Chapter 3: Dynamic Programming; policy evaluation, policy improvement, policy iteration, and value iteration

Hoda Zamani

PhD in Artificial Intelligence and Robotics

GitHub: [hodazamani.github.io](https://github.com/hodazamani)

[Hoda Zamani - Google Scholar](#)

Primary Textbook

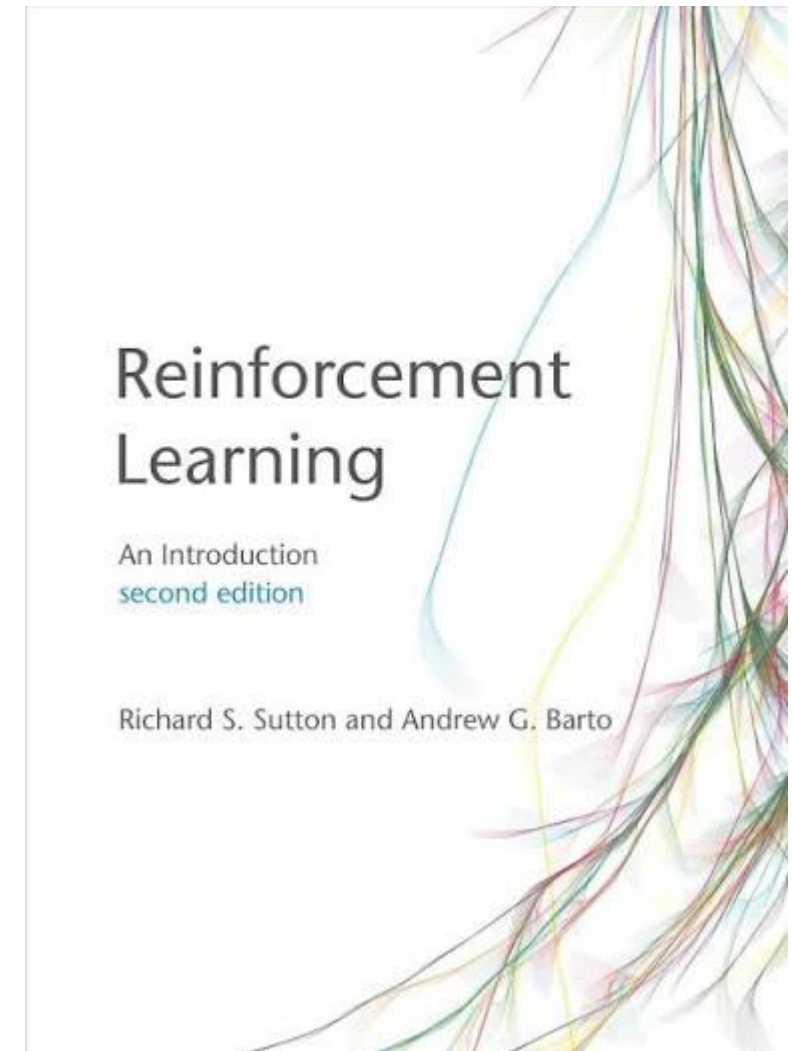
Reinforcement Learning: An Introduction, Second edition, in progress
Richard S. Sutton and Andrew G. Barto, [Download from: SuttonBartoIPRLBook2ndEd.pdf](#)

Recommended References

- Szepesvári, C. (2010). Algorithms for Reinforcement Learning. Morgan & Claypool Publishers.
- Bertsekas, D. P. (2019). Reinforcement Learning and Optimal Control. Athena Scientific.
- Puterman, M. L. (2014). Markov Decision Processes: Discrete Stochastic Dynamic Programming. Wiley.

Additional Course Materials

- Instructor-provided lecture notes, slides, and supplementary reading materials.
- Programming assignments and practical exercises using Python and relevant machine learning libraries.
- Selected research papers and technical articles on reinforcement learning and decision-making.
- Computational demonstrations, datasets, and examples provided throughout the course.



You can find:

-  Course announcements & communication
-  Slides & weekly schedule
-  Textbook & reading materials
-  TA
-  Course policies

Here: <https://hodazamani.github.io/Courses/>

Assessment Component	Weight
Weekly Assignments & Programming Exercises	20%
Midterm Examination	30%
Reinforcement Learning Project/Presentation	20%
Final Examination	30%
Total	100%

CONTENTS

Policy Evaluation

Policy Improvement

Policy Iteration

Value Iteration

Asynchronous Dynamic Programming

Generalized Policy Iteration

Efficiency of Dynamic Programming

Definition

- Dynamic Programming (DP) is a family of algorithms used to find **optimal policies** in a **Markov Decision Process (MDP)**.
- DP assumes that the environment is **fully known**.
- It uses the known transition probabilities and rewards to calculate **value functions**.
- DP is an important theoretical foundation for **Reinforcement Learning (RL)**.

Main idea:

Use value functions to systematically search for the best actions/policy.

Classical DP has two major limitations:

1. Requires a perfect model

- Transition probabilities must be known:

$$p(s', r | s, a)$$

2. Computationally expensive

- Large state and action spaces require many calculations.

Reinforcement Learning tries to achieve the same goal:

Find optimal policy

but **without requiring a perfect model** and often with **less computation**.

For DP, we commonly assume a **finite MDP**.

An MDP consists of:

- $\mathcal{S} \rightarrow$ set of states
- $A(s) \rightarrow$ available actions in state s
- $R \rightarrow$ possible rewards
- $p(s', r | s, a) \rightarrow$ probability of moving to s' and receiving reward r

The environment dynamics are:

$$p(s', r | s, a) = P(S_{t+1} = s', R_{t+1} = r | S_t = s, A_t = a)$$

Read it from left to right:

$$p(\underbrace{s', r}_{\text{next state and reward}} | \underbrace{s, a}_{\text{current state and action}})$$

So:

Current State + Action $\rightarrow$ Next State + Reward

A Robot Moving Between Two Rooms

Question:

A robot can be in two states, A and B . In each state, the robot can choose either **Stay** or **Move**.

- If the robot is in state A and chooses **Move**, it moves to B with probability 1 and receives a reward of 5.
- If the robot is in state B and chooses **Move**, it moves to A with probability 1 and receives a reward of 10.
- The robot always chooses **Move**.
- The discount factor is $\gamma = 0.9$.

Calculate $V^\pi(A)$ and $V^\pi(B)$.

Step 1: Identify what is given

Step 2: What are we being asked?

A Robot Moving Between Two Rooms

Question:

A robot can be in two states, A and B . In each state, the robot can choose either **Stay** or **Move**.

- If the robot is in state A and chooses **Move**, it moves to B with probability 1 and receives a reward of 5.
- If the robot is in state B and chooses **Move**, it moves to A with probability 1 and receives a reward of 10.
- The robot always chooses **Move**.
- The discount factor is $\gamma = 0.9$.

Calculate $V^\pi(A)$ and $V^\pi(B)$.

Step 1: Identify what is given

1. States

The robot has two states:

$$S = \{A, B\}$$

2. Actions

The available actions are:

$$A(s) = \{\text{Stay}, \text{Move}\}$$

But the question says the robot **always chooses Move**.

Therefore:

$$\pi(\text{Move}|A) = 1$$

$$\pi(\text{Move}|B) = 1$$

3. Transition probabilities

From A , Move takes the robot to B :

$$p(B|A, \text{Move}) = 1$$

From B , Move takes the robot to A :

$$p(A|B, \text{Move}) = 1$$

4. Rewards

When moving from A to B :

$$R = 5$$

When moving from B to A :

$$R = 10$$

5. Discount factor

$$\gamma = 0.9$$

Step 2: What are we being asked?

The question asks us to calculate:

$$V^{\pi}(A)$$

and

$$V^{\pi}(B)$$

Because the **policy is already given** — always choose Move — we use the **Bellman expectation equation**.

The general Bellman expectation equation is:

$$V^{\pi}(s) = \sum_a \pi(a|s) \sum_{s', r} p(s', r|s, a) [r + \gamma V^{\pi}(s')]$$

Step 4: Calculate $V^\pi(A)$

Start with the general equation:

$$V^\pi(A) = \sum_a \pi(a|A) \sum_{s',r} p(s',r|A,a)[r + \gamma V^\pi(s')]$$

The robot always chooses **Move**, so:

$$\pi(\text{Move}|A) = 1$$

And:

$$p(B, 5|A, \text{Move}) = 1$$

Therefore:

$$V^\pi(A) = 1 \times 1 [5 + 0.9V^\pi(B)]$$

So:

$$V^\pi(A) = 5 + 0.9V^\pi(B)$$

Step 5: Calculate $V^\pi(B)$

Again, start with the formula:

$$V^\pi(B) = \sum_a \pi(a|B) \sum_{s',r} p(s',r|B,a)[r + \gamma V^\pi(s')]$$

The robot always chooses Move:

$$\pi(\text{Move}|B) = 1$$

And:

$$p(A, 10|B, \text{Move}) = 1$$

Therefore:

$$V^\pi(B) = 1 \times 1 [10 + 0.9V^\pi(A)]$$

So:

$$V^\pi(B) = 10 + 0.9V^\pi(A)$$

Step 6: Now solve the two equations

We have:

$$V^\pi(A) = \underbrace{5}_{\text{reward}} + \underbrace{0.9}_{\text{discount factor}} \underbrace{V^\pi(B)}_{\text{next-state value}}$$

and

$$V^\pi(B) = \underbrace{10}_{\text{reward}} + \underbrace{0.9}_{\text{discount factor}} \underbrace{V^\pi(A)}_{\text{next-state value}}$$

Now substitute the second equation into the first.

Substitution

$$V^\pi(A) = 5 + 0.9 \underbrace{[10 + 0.9V^\pi(A)]}_{V^\pi(B)} \quad V^\pi(A) = 5 + 0.9V^\pi(B)$$

Replace $V^\pi(B)$ with:

$$10 + 0.9V^\pi(A)$$

Therefore:

$$V^\pi(A) = 5 + 0.9[10 + 0.9V^\pi(A)]$$

Expand:

$$V^\pi(A) = 5 + 9 + 0.81V^\pi(A)$$

$$V^\pi(A) = 14 + 0.81V^\pi(A)$$

Move $0.81V^\pi(A)$ to the left:

$$V^\pi(A) - 0.81V^\pi(A) = 14$$

$$0.19V^\pi(A) = 14$$

Divide by 0.19:

$$V^\pi(A) = \frac{14}{0.19}$$

Therefore:

$$V^\pi(A) = 73.68$$

Step 7: Find $V^\pi(B)$

We already know:

$$V^\pi(B) = 10 + 0.9V^\pi(A)$$

Substitute:

$$V^\pi(B) = 10 + 0.9(73.68)$$

$$V^\pi(B) = 10 + 66.312$$

Therefore:

$$V^\pi(B) = 76.312$$

Approximately:

$$V^\pi(B) \approx 76.32$$

Final Answer

$$V^\pi(A) = 73.68$$

$$V^\pi(B) = 76.32$$

The key idea of DP is to use **value functions** to evaluate states and actions.

Two important value functions are:

State-value function:

$$v_{\pi}(s) = E_{\pi}[G_t | S_t = s]$$

It tells us:

"How good is it to be in state s ?"

Action-value function:

$$q_{\pi}(s, a) = E_{\pi}[G_t | S_t = s, A_t = a]$$

It tells us:

"How good is it to take action a in state s ?"

Once we know the **optimal value functions**, we can obtain the optimal policy.

$v_{\pi}(s)$ The value of being in state s , when following policy π .

E_{π} Expected value when following policy π .

G_t $G_t = R_{t+1} + \gamma R_{t+2} + \gamma^2 R_{t+3} + \dots$

$S_t = s$ We are currently in state s .

$$v_{\pi}(s) = E_{\pi}[G_t | S_t = s]$$

If I am currently in state s and I follow policy π , how much total future reward do I expect to receive?

Bellman Optimality Equation for v^*

The optimal state-value function is:

$$v^*(s) = \max_a E[R_{t+1} + \gamma v^*(S_{t+1}) \mid S_t = s, A_t = a]$$

Using the transition probabilities:

$$v^*(s) = \max_a \sum_{s', r} p(s', r \mid s, a) [r + \gamma v^*(s')]$$

Meaning

For every state:

Optimal value = best action's expected immediate reward + discounted future value

Bellman Optimality Equation for q^*

The optimal action-value function is:

$$q^*(s, a) = E[R_{t+1} + \gamma \max_{a'} q^*(S_{t+1}, a') \mid S_t = s, A_t = a]$$

Using the transition model:

$$q^*(s, a) = \sum_{s', r} p(s', r \mid s, a) \left[r + \gamma \max_{a'} q^*(s', a') \right]$$

Key idea

After taking action a :

1. Receive reward r
2. Move to state s'
3. Choose the best future action
4. Discount the future value by γ

The Bellman equation can be converted into an **update rule**.

For state values:

$$V_{\text{new}}(s) = \max_a \sum_{s', r} p(s', r | s, a) [r + \gamma V_{\text{old}}(s')]$$

This is called **Value Iteration**.

The process is repeated:

$$V_0 \rightarrow V_1 \rightarrow V_2 \rightarrow \dots \rightarrow V^*$$

As the values converge:

$$V_k(s) \rightarrow V^*(s)$$

After the values converge, choose the action with the highest expected return:



Finally:

$$\pi^*(s) = \arg \max_a \sum_{s', r} p(s', r | s, a) [r + \gamma V^*(s')]$$

DP repeatedly applies this update to improve the value estimates until they converge to the optimal values.

Suppose we have a fixed policy π . **Policy evaluation** means:

Given the policy π , how good is each state?

We want to calculate the **state-value function**

$$v_{\pi}(s) = \mathbb{E}_{\pi}[G_t \mid S_t = s]$$

where G_t is the total discounted future reward.

Using the Bellman equation:

$$v_{\pi}(s) = \sum_a \pi(a|s) \sum_{s',r} p(s',r|s,a) [r + \gamma v_{\pi}(s')]$$

In words:

Value of current state = expected immediate reward + expected discounted value of the next state.

If the environment is completely known, we could solve the Bellman equations directly.

For example, if there are 100 states, we'd have 100 simultaneous equations:

$$v_{\pi}(s_1), v_{\pi}(s_2), \dots, v_{\pi}(s_{100})$$

Solving them as a system of linear equations is possible, but can be inconvenient.

Then repeatedly apply:

$$v_{k+1}(s) = \sum_a \pi(a|s) \sum_{s',r} p(s',r|s,a)[r + \gamma v_k(s')]$$

So:

$$v_0 \rightarrow v_1 \rightarrow v_2 \rightarrow v_3 \rightarrow \dots \rightarrow v_\pi$$

Eventually,

$$v_k(s) \rightarrow v_\pi(s)$$

A **backup** means updating the value of a state using information about its possible successors.

For a particular state s :

$$s \rightarrow \{s'_1, s'_2, \dots\}$$

We look at **all possible next states**, calculate their values, add the immediate rewards, and take the expectation according to the policy and environment.

That's why this is called a **full backup**.

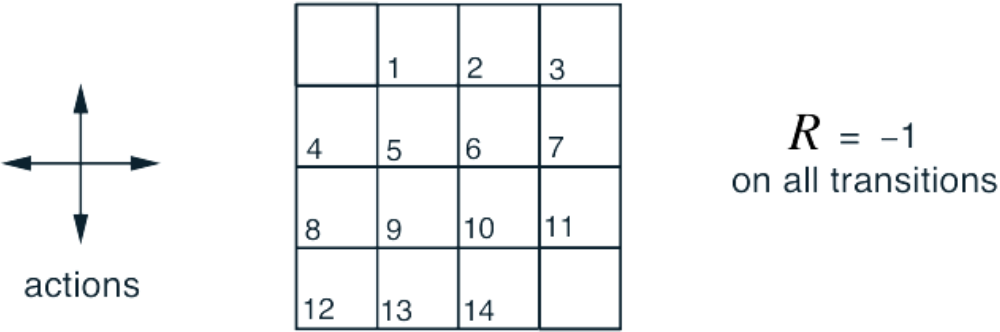
It is different from sampling one experience such as:

$$s \rightarrow a \rightarrow s'$$

because DP considers **every possible transition**.

Example — Iterative Policy Evaluation

Considers a 4x4 gridworld:



The terminal state has value:

$$\gamma = 1$$

$$V(T) = 0$$

There are 14 non-terminal states and one terminal state T .

Given

There are four actions:

$$A = \{Up, Down, Left, Right\}$$

The policy is random, so:

$$\pi(a|s) = \frac{1}{4}$$

for every action.

The reward is:

$$R = -1$$

Step 1 — Bellman equation

For this problem:

$$V_{k+1}(s) = \frac{1}{4} \sum_a [-1 + V_k(s')]$$

Therefore:

$$V_{k+1}(s) = -1 + \frac{V_k(s_U) + V_k(s_D) + V_k(s_L) + V_k(s_R)}{4}$$

If an action takes us outside the grid, we remain in the same state.

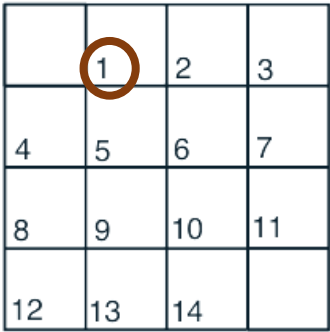
Step 2 — Initialize V_0

Initially:

$$V_0(s) = 0$$

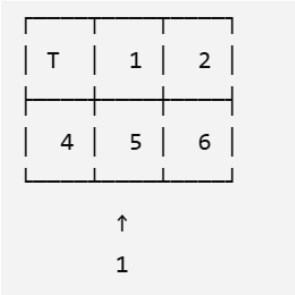
for all states.

```
+-----+
| T | 0 | 0 | 0 |
+-----+
| 0 | 0 | 0 | 0 |
+-----+
| 0 | 0 | 0 | 0 |
+-----+
| 0 | 0 | 0 | T |
+-----+
```



$R = -1$
on all transitions

Look at **state 1** in the grid:



Step 3 — Calculate V_1

Let's calculate a few states carefully.

State 1

From state 1:

- Up $\rightarrow T$
- Down $\rightarrow 5$
- Left $\rightarrow T$
- Right $\rightarrow 2$

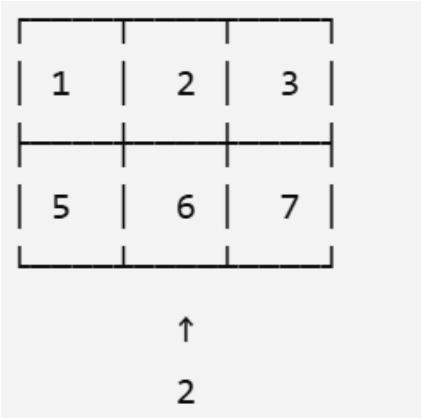
Therefore:

Action	Next state	V_0
Up	T	0
Down	5	0
Left	T	0
Right	2	0

$$\begin{aligned} V_1(1) &= -1 + \frac{V_0(T) + V_0(5) + V_0(T) + V_0(2)}{4} \\ &= -1 + \frac{0 + 0 + 0 + 0}{4} \\ &= -1 \end{aligned}$$

	1	2	3
4	5	6	7
8	9	10	11
12	13	14	

$R = -1$
on all transitions



State 2

From state 2:

- Up → 1
- Down → 6
- Left → 1
- Right → 3

Thus:

$$V_1(2) = -1 + \frac{0 + 0 + 0 + 0}{4}$$

$V_1(2) = -1$

Action	Next state	V_0
Up	2	0
Down	6	0
Left	1	0
Right	3	0

Example — Iterative Policy Evaluation

	1	2	3
4	5	6	7
8	9	10	11
12	13	14	

$R = -1$
on all transitions

Therefore:

State 3

From state 3:

- Up → 3
- Down → 7
- Left → 2
- Right → 3

2	3	
6	7	

Action	Next state	V_0
Up	3	0
Down	7	0
Left	2	0
Right	3	0

V_1

0.0	-1.0	-1.0	-1.0
-1.0	-1.0	-1.0	-1.0
-1.0	-1.0	-1.0	-1.0
-1.0	-1.0	-1.0	0.0

Again:

$V_1(3) = -1$

The same happens for every non-terminal state because all values in V_0 are zero.

	1	2	3
4	5	6	7
8	9	10	11
12	13	14	

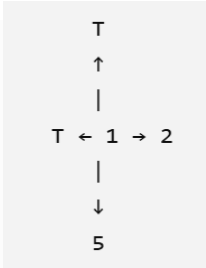
$R = -1$
on all transitions

Step 4 — Calculate V_2

Now we use V_1 .

State 1

The neighboring states are:



$T, 5, T, 2$

Their values are:

$0, -1, 0, -1$

Therefore:

0.0	-1.0	-1.0	-1.0
-1.0	-1.0	-1.0	-1.0
-1.0	-1.0	-1.0	-1.0
-1.0	-1.0	-1.0	0.0

$V_1(T) = 0$

$V_1(5) = -1$

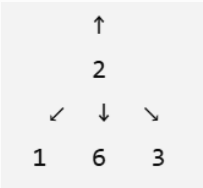
$V_1(2) = -1$

$$V_2(1) = -1 + \frac{0 + (-1) + 0 + (-1)}{4}$$
$$= -1 - \frac{2}{4}$$

$V_2(1) = -1.5$

State 2

Its successors are:



$2, 6, 1, 3$

All have value -1 in V_1 .

Therefore:

$$V_2(2) = -1 + \frac{-1 - 1 - 1 - 1}{4}$$

$V_2(2) = -2$

Action	Next state	V_1
Up	2	-1
Down	6	-1
Left	1	-1
Right	3	-1

State 3

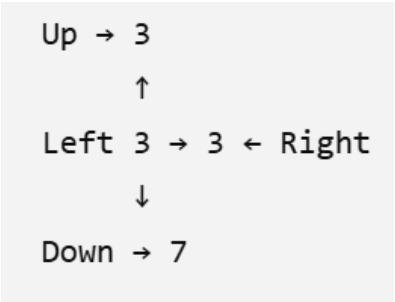
Its successors are:

3, 7, 2, 3

All have value -1 .

Therefore:

$V_2(3) = -2$



Action	Next state	V_1
Up	3	-1
Down	7	-1
Left	2	-1
Right	3	-1

Therefore:

$$V_2(3) = -1 + \frac{-1 - 1 - 1 - 1}{4}$$

$V_2(3) = -2$

	1	2	3
4	5	6	7
8	9	10	11
12	13	14	

$R = -1$
on all transitions

State 4

Successors:

- Up $\rightarrow$ 1
- Down $\rightarrow$ 8
- Left $\rightarrow$ 4
- Right $\rightarrow$ 5

All have value -1 .

Therefore:

$V_2(4) = -2$

State 14

This one is interesting.

From state 14:

- Up $\rightarrow$ 10
- Down $\rightarrow T$
- Left $\rightarrow$ 13
- Right $\rightarrow T$

Therefore:

$$\begin{aligned} V_2(14) &= -1 + \frac{V_1(10) + V_1(T) + V_1(13) + V_1(T)}{4} \\ &= -1 + \frac{-1 + 0 - 1 + 0}{4} \\ &= -1 - \frac{1}{2} \\ V_2(14) &= -1.5 \end{aligned}$$

Step 5 — Complete V_2

Therefore:

V_2

+-----+-----+-----+-----+
T -1.5 -2.0 -2.0
+-----+-----+-----+-----+
-2.0 -2.0 -2.0 -2.0
+-----+-----+-----+-----+
-2.0 -2.0 -2.0 -2.0
+-----+-----+-----+-----+
-2.0 -2.0 -1.5 T
+-----+-----+-----+-----+

Example — Iterative Policy Evaluation

Now we use V_2 .

For example, state 1:

$$\begin{aligned}
 V_3(1) &= -1 + \frac{V_2(T) + V_2(5) + V_2(T) + V_2(2)}{4} \\
 &= -1 + \frac{0 - 2 + 0 - 2}{4} \\
 &= -1 - \frac{4}{4} \\
 \boxed{V_3(1) = -2}
 \end{aligned}$$

	1	2	3
4	5	6	7
8	9	10	11
12	13	14	

$R = -1$
on all transitions

For state 2:

$$\begin{aligned}
 V_3(2) &= -1 + \frac{V_2(2) + V_2(6) + V_2(1) + V_2(3)}{4} \\
 &= -1 + \frac{-2 - 2 - 1.5 - 2}{4} \\
 &= -1 - \frac{7.5}{4} \\
 \boxed{V_3(2) = -2.875}
 \end{aligned}$$

Similarly, we continue this calculation for every state.

We keep repeating:

$$V_0 \rightarrow V_1 \rightarrow V_2 \rightarrow V_3 \rightarrow \dots$$

Eventually, the values converge to the true value function:

$$\boxed{V(s) \rightarrow v_\pi(s)}$$

The final value represents the **negative expected number of steps until reaching the terminal state**.

For example, if:

$$v_\pi(s) = -10$$

that means that, under the random policy, the expected number of steps to termination is approximately

$$10$$

because every step gives a reward of -1 .

For this specific Gridworld:

$$\boxed{V_{k+1}(s) = -1 + \frac{1}{4} [V_k(s_U) + V_k(s_D) + V_k(s_L) + V_k(s_R)]}$$

We calculate this for **every state**, repeatedly, until:

$$\boxed{\max_s |V_{k+1}(s) - V_k(s)| < \theta}$$

Then we stop and say:

$$\boxed{V \approx v_\pi}$$

This is **Iterative Policy Evaluation**.

We already have a policy π .

We also calculated its value function:

$$v_{\pi}(s)$$

This tells us:

“If I start in state s and continue following policy π , how good is that?”

Now we ask:

“Could I make the policy better by choosing a different action?”

That is exactly what **policy improvement** does.

We first calculate its value function $v_{\pi}(s)$.

Then, we check whether another action can produce a higher value.

- The value of taking action a in state s and then following π is:

$$q_{\pi}(s, a) = E_{\pi}[R_{t+1} + \gamma v_{\pi}(S_{t+1})]$$

- If

$$q_{\pi}(s, a) > v_{\pi}(s)$$

then action a is better than the current action.

- If a new policy π' satisfies:

$$q_{\pi}(s, \pi'(s)) \geq v_{\pi}(s)$$

for every state s , then:

$$v_{\pi'}(s) \geq v_{\pi}(s)$$

- Therefore, the new policy cannot be worse.
- If the inequality is strict somewhere, the new policy is strictly better.

- Repeat **policy evaluation** → **policy improvement**.
- When no further improvement is possible:

$$v_{\pi}(s) = \max_a E[R_{t+1} + \gamma v_{\pi}(S_{t+1})]$$

- This is the **Bellman Optimality Equation**.
- Therefore, the policy is **optimal**.

Greedy Policy

- Choose the action with the highest $q_{\pi}(s, a)$:

$$\pi'(s) = \arg \max_a q_{\pi}(s, a)$$

- This is called a **greedy policy**.
- It selects the action that looks best according to the current value function.
- Greedy improvement guarantees a better or equal policy.

- Repeat **policy evaluation** → **policy improvement**.
- When no further improvement is possible:

$$v_{\pi}(s) = \max_a E[R_{t+1} + \gamma v_{\pi}(S_{t+1})]$$

- This is the **Bellman Optimality Equation**.
- Therefore, the policy is **optimal**.

- **Policy Iteration** repeatedly improves a policy until it becomes optimal.
- Two main steps:
 1. **Policy Evaluation:** Calculate v_π .
 2. **Policy Improvement:** Make the policy greedy with respect to v_π .

$$\pi_0 \xrightarrow{E} v_{\pi_0} \xrightarrow{I} \pi_1 \xrightarrow{E} v_{\pi_1} \xrightarrow{I} \dots \rightarrow \pi^*$$

- Each new policy is guaranteed to be **better than the previous one**, unless it is already optimal.

1. Initialization

- Start with arbitrary $V(s)$ and policy $\pi(s)$.

2. Policy Evaluation

- Repeatedly update:

$$V(s) \leftarrow \sum_{s',r} p(s',r|s,\pi(s)) [r + \gamma V(s')]$$

- Stop when V changes by only a small amount.

3. Policy Improvement

- Choose the best action:

$$\pi(s) \leftarrow \arg \max_a \sum_{s',r} p(s',r|s,a) [r + \gamma V(s')]$$

- If the policy does not change → **Stop: optimal policy.**

- A finite MDP has a **finite number of possible policies**.
- Every policy improvement produces a better policy.
- Therefore, the algorithm must eventually converge to:

$$\boxed{\pi^*, v^*}$$

- In practice, Policy Iteration often needs **surprisingly few iterations**.
- Example: In some problems, the optimal policy can be found after only **one improvement step**.

Problem with Policy Iteration:

Policy evaluation may require many iterations.

Value Iteration solves this by performing only **one evaluation sweep** before improving again.

$$V_{k+1}(s) = \max_a E[R_{t+1} + \gamma V_k(S_{t+1})]$$

- It directly applies the **Bellman Optimality Equation**.
- No separate policy evaluation is required.
- Stop when the change in V becomes very small.

$$\Delta < \theta$$

Policy Iteration	Value Iteration
Evaluate policy	Update value directly
Improve policy	Take maximum over actions
Evaluation may need many sweeps	One sweep per iteration
Alternates Evaluation → Improvement	Combines both steps
Converges to optimal policy	Converges to v^*

- Traditional DP methods require **sweeps over all states**.
- This becomes impractical for very large state spaces.
- **Asynchronous DP** updates states in **any order**, without complete sweeps.
- Some states may be updated many times before others are updated once.
- To converge, **every state must continue to be updated**.

$$V(s_k) \leftarrow \max_a E[R + \gamma V(S')]$$

Key idea: Update the most useful states first instead of waiting for a full sweep.

- Provides flexibility in choosing **which states to update**.
- Can focus computation on states that are most relevant to the agent.
- Value information can propagate more efficiently between states.
- Can combine computation with **real-time interaction**.
- The agent's experience can determine which states should be updated.

Important: Avoiding full sweeps does not necessarily reduce total computation; it allows computation to be **better focused**.

GPI describes the interaction between two processes:

Policy Evaluation

$$V \rightarrow V^\pi$$

Policy Improvement

$$\pi \rightarrow \textit{greedy}(V)$$

- Evaluation makes the value function consistent with the current policy.
- Improvement makes the policy greedy with respect to the value function.
- These processes can be performed alternately or **interleaved**.

Evaluation $\leftrightarrow$ Improvement

When both stabilize:

$V = V^*, \quad \pi = \pi^*$

- DP can be very efficient compared with direct search.
- Its computational complexity is **polynomial** in the number of states and actions.
- This is much better than searching through all possible policies.
- The main challenge is the **curse of dimensionality**: state spaces can grow exponentially with the number of variables.
- For very large problems, **Asynchronous DP** and **GPI** can be more practical because they focus computation on relevant states.

Question?

Thank you!